How Database Connection Pooling Works

Opening a new database connection is expensive. Each connection requires a network handshake, authentication, and memory allocation on the database server. A connection pool avoids repeating this work by keeping a set of connections open and handing them out as needed.

When an application needs to run a query, it borrows a connection from the pool, uses it, and returns it. If every connection is in use, new requests wait in a queue until one becomes free or a timeout expires. Pool size is therefore a trade-off: too small and requests queue up, too large and the database spends its memory and CPU managing idle connections.

Common settings include the minimum and maximum pool size, the maximum time a request may wait for a connection, and how long an idle connection may stay open before it is closed. Many pools also validate a connection before handing it out, which catches connections that the server has silently dropped.

Typical failure modes include connection leaks, where code borrows a connection and never returns it, gradually exhausting the pool, and thundering herds, where many clients reconnect at once after an outage. Monitoring the number of active, idle and waiting connections is usually the first step in diagnosing pool problems.